Agentic Context Engineering

Evolving Contexts for Self-Improving Language Models

Zhang, Hu, Upasani, Ma, Hong, Kamanuru, Rainton, Wu, Ji, Li, Thakker, Zou, Olukotun
Stanford University · SambaNova Systems · UC Berkeley

2026-01-01

Motivation: Context Adaptation for LLMs

Why adapt through *context* instead of *weights*?

Modern LLM applications (agents, compound AI systems) improve through **context adaptation** — modifying inputs with instructions, strategies, or evidence, rather than weight updates.

- **Interpretable & explainable** for users and developers — strategies are in natural language
- **Rapid integration** of new knowledge at runtime
- **Shareable** across models or modules in a compound system
- **Selective unlearning** — contexts are human-interpretable, so entries can be individually removed due to privacy or legal constraints (GDPR, CCPA)
- Increasingly practical via **long-context LLMs** and **KV-cache reuse**

Prior Work

Method	Approach
Reflexion	Reflect on failures to improve planning
TextGrad	Gradient-like textual feedback
GEPA	Genetic-Pareto prompt evolution
Dynamic Cheatsheet	Accumulate strategies in external memory

The Central Idea

> Contexts should function not as concise summaries, but as **comprehensive, evolving playbooks** — detailed, inclusive, and rich with domain insights.

Unlike humans who benefit from concise generalization, LLMs perform better with **long, detailed contexts** and can distill relevance autonomously at inference time.

Two Key Limitations of Existing Methods

1. Brevity Bias

Prompt optimizers prioritize **concise, broadly applicable** instructions, dropping critical domain-specific detail.

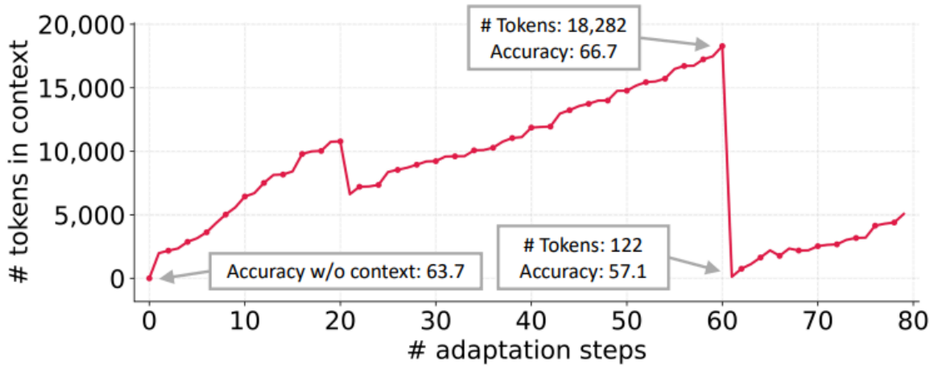
- GEPA highlights brevity as a strength, but abstraction can **omit heuristics, tool-use guidelines**, and common failure modes
- Iterative methods converge to near-identical instructions (e.g., *"Create unit tests to ensure methods behave as expected"*)
- Sacrifices diversity and propagates recurring errors

The Problem

Short prompts miss the detailed strategies required by agents and knowledge-intensive applications.

2. Context Collapse

Monolithic rewriting by an LLM **compresses** accumulated context into shorter, less informative summaries over time.

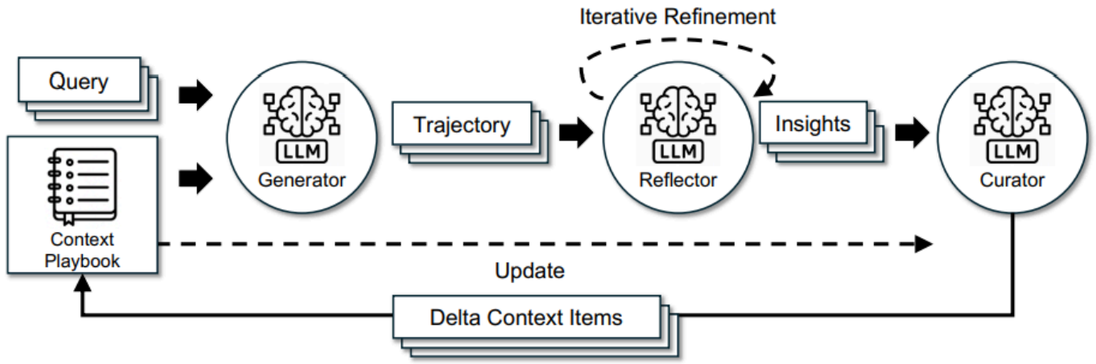


Context collapse on AppWorld

A single rewriting step erased accumulated knowledge (18,282 → 122 tokens) and dropped accuracy **below the unadapted baseline**.

The ACE Framework

Agentic Architecture with Three Specialized Roles



1. **Generator** — produces reasoning trajectories, highlights which bullets were useful or misleading
2. **Reflector** — critiques traces to extract concrete lessons; can iterate multiple refinement rounds
3. **Curator** — synthesizes lessons into compact **delta entries**, merged into the context by lightweight non-LLM logic

Three Key Innovations

① **Dedicated Reflector** Separates evaluation/insight extraction from curation, improving context quality

② **Incremental Delta Updates** Replaces costly monolithic rewrites with **localized edits**, preserving past knowledge

③ **Grow-and-Refine Mechanism** Balances steady context expansion with redundancy control via semantic de-duplication

Design Principles

- Contexts are **structured, itemized bullets** (not a monolithic prompt)
- Each bullet has metadata: unique ID + helpful/harmful counters
- Multiple deltas can be merged **in parallel** (batched adaptation)
- Supports **multi-epoch** adaptation over training data

Incremental Updates & Grow-and-Refine

Incremental Delta Updates

Instead of regenerating contexts in full, ACE produces **compact delta contexts**: small sets of candidate bullets distilled by the Reflector and integrated by the Curator.

Each bullet contains:

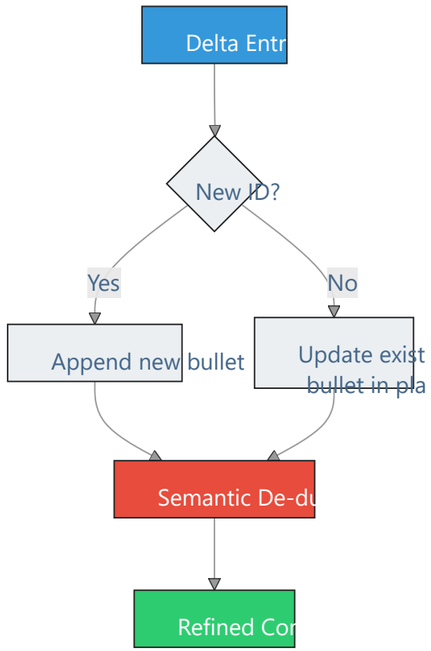
- **Metadata** — unique identifier + usage counters (helpful/harmful)
- **Content** — a reusable strategy, domain concept, or common failure mode

Three key properties:

Property	Benefit
Localization	Only relevant bullets are updated
Fine-grained retrieval	Generator focuses on most pertinent knowledge
Incremental adaptation	Efficient merging, pruning, de-duplication

- **New bullets** appended; **existing bullets** updated in place
- **Semantic de-duplication** prunes redundancy via embeddings
- Refinement: **proactive** (after each delta) or **lazy** (when context window exceeded)
- Contexts **expand adaptively** and avoid variance of monolithic rewriting

Grow-and-Refine



Results: Agent Benchmark (AppWorld)

Offline & Online Adaptation Results

Offline Adaptation

Method	GT Labels	Test-Normal (TGC)	Test-Challenge (TGC)	Average
ReAct (base)	—	63.7	41.5	42.4
+ ICL	✓	64.3	46.0	46.0
+ GEPA	✓	64.9	46.0	46.4
+ ACE	✓	**76.2** (+12.5)	**57.3** (+15.8)	**59.4** (+17.0)
+ ACE	✗	**75.0** (+11.3)	**54.4** (+12.9)	**57.2** (+14.8)

Online Adaptation

Method	GT Labels	Test-Normal (TGC)	Test-Challenge (TGC)	Average
+ DC (CU)	✗	65.5	52.3	51.9
+ ACE	✗	**69.6** (+5.9)	**66.0** (+24.5)	**59.5** (+17.1)

Key Findings

- ACE outperforms baselines by **+10.6%** average across all settings
- Works effectively **without ground-truth labels** — uses execution feedback alone
- On harder **test-challenge** split, online ACE achieves **+24.5%** over base

AppWorld Leaderboard

ACE matches the **#1-ranked** production agent (IBM CUGA, powered by GPT-4.1) on overall average, and **surpasses it** on the harder test-challenge split — using a **smaller open-source model** (DeepSeek-V3.1).

Self-Improving Agents

ACE enables agents to self-improve by dynamically refining context from execution feedback alone, **without labeled supervision**.

Results: Financial Analysis Benchmarks

Domain-Specific Reasoning (FiNER + Formula)

Offline Adaptation

Method	GT Labels	FiNER (Acc)	Formula (Acc)	Average
Base LLM	—	70.7	67.5	69.1
ICL	✓	72.3	67.0	69.6
MIPROv2	✓	72.4	69.5	70.9
GEPA	✓	73.5	71.5	72.5
ACE	✓	**78.3** (+7.6)	**85.5** (+18.0)	**81.9** (+12.8)
ACE	✗	71.1	83.0	77.1 (+8.0)

Online Adaptation

Method	GT Labels	FiNER (Acc)	Formula (Acc)	Average
DC (CU)	✓	74.2	69.5	71.8
DC (CU)	✗	68.3	62.5	65.4
ACE	✓	**76.7** (+6.0)	76.5	**76.6** (+7.5)
ACE	✗	67.3	**78.5**	72.9 (+3.8)

Key Findings

- ACE surpasses all baselines by **+8.6% average** with GT labels in offline setting
- On Formula, ACE achieves a remarkable **+18.0%** gain (67.5 → 85.5)
- Structured, evolving contexts are especially effective for **precise domain knowledge** (financial concepts, XBRL rules)

Limitation: Feedback Quality Matters

Without reliable supervision or execution signals, both ACE and Dynamic Cheatsheet can **degrade** — the constructed context gets polluted by spurious signals.

Warning

ACE is most effective when **reliable feedback** is available (e.g., code execution results, formula correctness). Unsupervised adaptation in domains lacking natural feedback signals remains challenging.

Efficiency: Cost & Latency Analysis

Offline Adaptation (AppWorld)

Metric	GEPA	ACE	Reduction
Latency (s)	53,898	9,517	**−82.3%**
# Rollouts	1,434	357	**−75.1%**

Online Adaptation (FiNER)

Metric	DC (CU)	ACE	Reduction
Latency (s)	65,104	5,503	**−91.5%**
Token Cost ($)	$17.7	$2.9	**−83.6%**

Average: 86.9% lower adaptation latency

ACE achieves higher accuracy with dramatically lower compute cost and wall-clock time.

Why ACE Is Efficient

1. **Incremental delta updates** — no full-context regeneration
2. **Non-LLM-based merging** — de-duplication and merge logic is deterministic, not neural
3. **Parallel delta merging** — multiple updates can be integrated simultaneously
4. **Fewer rollouts needed** — structured reflection extracts more signal per sample

Ablation: What Matters Most?

Variant	Avg. Accuracy
ACE w/o Reflector or multi-epoch	55.1 (+12.7)
ACE w/o multi-epoch	56.8 (+14.4)
Full ACE	**59.4 (+17.0)**
+ offline warmup (online)	**59.5 (+17.1)**

Every component contributes: the **Reflector** adds +1.7%, **multi-epoch** adds +2.6%, and **offline warmup** further boosts online performance.

Takeaways & Discussion

Key Contributions

1. **ACE framework** — treats contexts as evolving playbooks with generation, reflection, and curation
2. **Solves brevity bias & context collapse** via incremental delta updates and grow-and-refine
3. **Strong empirical results** — +10.6% on agents, +8.6% on finance, matching #1 on AppWorld leaderboard with smaller open-source model
4. **Self-improving without labels** — leverages natural execution feedback
5. **86.9% lower latency** and dramatically reduced costs vs. existing methods

Broader Implications

- **Longer context ≠ higher cost** — KV-cache reuse, compression, and offloading make long contexts practical
- **Alternative to fine-tuning** for continuous/online learning — cheaper, interpretable, supports selective unlearning
- **Privacy-aware** — context entries can be individually removed (GDPR, CCPA compliance)

Limitations

- Relies on a **reasonably strong Reflector** — if the model cannot extract useful insights, context quality degrades
- Not all tasks benefit: simple Q&A or fixed-strategy tasks need only concise prompts
- **Feedback quality is critical** — without reliable signals, adaptation can introduce noise

Future Directions

- Extend ACE to **more domains** (legal, medical, scientific reasoning)
- Investigate **cross-task context transfer** — can playbooks generalize?
- Integrate with **retrieval-augmented generation** for hybrid context construction
- Combine with **weight adaptation** (LoRA, prefix tuning) for complementary gains

Resources

- Paper: arXiv:2510.04618
- Code: github.com/ace-agent/ace
- Website: ace-agent.github.io